Imaxes/Simbolo_logo_cor transp.png

Grado en Intelixencia Artificial
Sistemas Multiaxente

Práctica 2: Sistemas Multiagente

Ainhoa de Diego Silva
Ander Asteinza García
Rodrigo Romay Aba
Grupo de clase: martes

Mundo Virtual

Al principio del proyecto enfocamos el mundo virtual como un mapa predeterminado de Unity. Sin embargo, para que la ejecución no ocupara mucho almacenamiento local y tuviera un flujo rápido de carga, decidimos descargar un paquete de Unity que contenía elementos prefabricados para realizar un mapa desde cero. Utilizamos un mapa de ejemplo de "DungeonsDragons" pero lo adaptamos para las necesidades de nuestro mundo. Elegimos ese esquema para utilizar como guía ya que tenía bastantes habitaciones para lograr que el mapa fuera más extenso y una disposición interesante para la ubicación de los agentes.

En la primera entrega, nuestro entorno se basa en una especie de laberinto con pasillos y algunos arcos interconectados para poder burlar la seguridad de los agentes. Cabe destacar un área central con una columna, lugar de máxima dificultad ya que los agentes confluyen en sus patrullas. En la habitación a

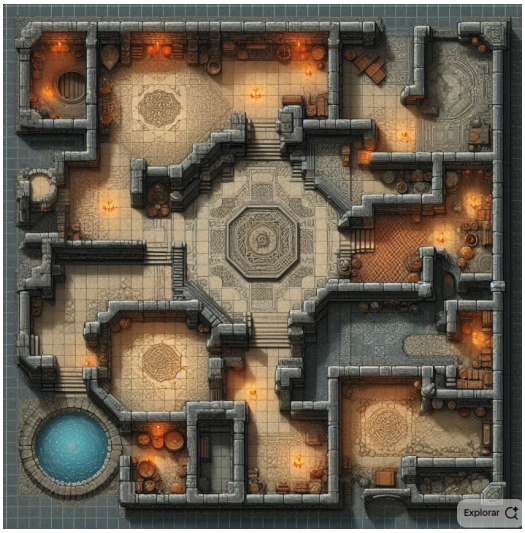

Figure 1: Mapa de referencia utilizado

la derecha del mapa estará el tesoro a conseguir, custodiado por otro agente. En el momento en el que se obtenga ese item, se abrirá la puerta del inicio para dar fin a la ejecución.

Figure 2: Mapa de nuestro mundo virtual versión 1

En esta segunda entrega del proyecto, introducimos varias modificaciones al mapa. Primeramente, ampliamos las dimensiones de las habitaciones y añadimos una habitación intermedia entre el pasillo inferior para poder acceder directamente a una de las salas donde se encuentra el tesoro. Además, introducimos puertas dinámicas (que únicamente se escuchan cuando las abre el jugador) y añadimos más arcos para tener más salidas de escape. Esta decisión se basó en la intención de facilitar un poco más al jugador ganar la

partida, ya que la dificultad aumentará con la comunicación de los agentes.

Figure 3: Mapa de nuestro mundo virtual versión 2

Hemos de destacar también que ahora el agente puede perseguir al jugador hasta la zona de salida, elevando así un poco la dificultad del juego.

Probando de manera repetida la funcionalidad, nos percatamos que con un único cofre, los fantasmas realizaban un recorrido que era muy complicado de burlar hacia la salida. Debido a esto, decidimos introducir un segundo cofre del tesoro para que los agentes se repartieran de manera más equitativa. Por lo tanto, el objetivo del juego será robar los dos tesoros (en ese momento se abrirá la puerta del inicio) y escapar del escenario sin que los agentes intercepten al jugador.

En cuanto a los agentes, en esta entrega, introducimos a mayores tres cámaras de seguridad (en el pasillo intermedio, en la primera sala del tesoro y en el pasillo que lleva a la segunda sala del tesoro). Éstas se basan solamente en la vista, es decir, funcionan de la siguiente manera: si no han detectado al objetivo presentan una luz verde, en cambio, si el jugador entra en cierto ángulo de visión, la luz se pone roja y la cámara se encarga de avisar a los agentes de que acaba de ver al jugador, para que así ellos se enacrguen de repartirse para lograr atraparlo.

Arquitectura individual

Para los fantasmas hemos adoptado una arquitectura híbrida inspirada en el modelo BDI.

- Capa Reactiva: Está compuesta por un sistema de sensores independientes (SensorVision, SensorEscucha, SensorContacto) que permiten

al agente percibir los cambios del entorno en tiempo real de forma autónoma.

- Capa Deliberativa: Centralizada en el componente CerebroGhost. Este componente mantiene las "creencias" del agente (como la posición actual del ladrón o su distancia a los objetivos) y procesa las intenciones a través de una Máquina de Estados Finitos (Patrullar, Vigilar, Perseguir, Confundido, Investigar).

Las comunicaciones se incorporan directamente en el ciclo de actualización de la capa deliberativa: los agentes cuentan con los recursos para recibir y gestionar mensajes de forma asíncrona. Estos se almacenan en una cola (buzon) y en un historial (historialConversaciones) que permite a los agentes almacenar y procesar conversaciones completas.

Por otro lado, los agentes Cámara poseen una arquitectura puramente reactiva. Su único propósito es percibir el entorno mediante la visión y actuar como desencadenantes (mediante peticiones de ayuda o Requests) sin capacidad de movimiento ni deliberación compleja.

Comunicación entre agentes

La comunicación del sistema se basa estrictamente en el lenguaje de comunicación FIPA-ACL y sus protocolos. Los mensajes intercambiados se instancian a través de la clase MensajeACL, la cual implementa los siguientes parámetros: performativa, sender, receiver y content. Adicionalmente, se utilizan los parámetros conversationId, replyWith e inReplyTo para la gestión de las conversaciones y el control de las subastas.
El protocolo de comunicación principal implementado es el Contract Net, utilizado para la distribución cooperativa de tareas. Su ejecución sigue el siguiente flujo:

1. Reconocimiento y Anuncio: Un agente asume el rol de "Gestor" temporal y emite un mensaje en broadcast con la performativa cfp (Call for Proposal) detallando las coordenadas del jugador.

2. Licitación: Los agentes receptores evalúan su disponibilidad y responden con propose (incluyendo sus heurísticas de distancia) o con refuse si están ocupados persiguiendo.

3. Adjudicación: Tras un tiempo límite de 0.5 segundos, el Gestor evalúa las propuestas y finaliza la negociación enviando accept-proposal a los agentes mejor posicionados y reject-proposal al resto.

Formato del contenido de los lenguajes

Hemos diseñado un lenguaje de contenido estructurado en strings, empaquetando múltiples variables numéricas.
El contenido de los mensajes se estructura utilizando el carácter almohadilla (#) como delimitador posicional de los datos. De esta forma:

- En los anuncios de alarma (Cfp), el contenido sigue el esquema : `Identificador#CoordenadaX#CoordenadaY#CoordenadaZ` (ej. Subasta#10.5#0#-4.2).
- En las pujas de los contratistas (Propose), el contenido empaqueta las métricas de aptitud del agente estructuradas como: `DistanciaCofre1#DistanciaCofre2#DistanciaSalida`

El procesamiento de este lenguaje se realiza en el método AnalizarMensaje. Mediante funciones de parsing, el agente divide la cadena de texto, extrae los valores numéricos y los convierte a tipos de datos espaciales (Vector3 para posiciones, float para distancias), permitiendo que el texto se transforme en directrices de navegación directa para los fantasmas.

Estrategia Multiagente

La estrategia cooperativa implementada garantiza que los agentes no formen un equipo predefinido, sino que su cooperación surja dinámicamente durante la ejecución del programa ante la detección del ladrón.

El objetivo global del sistema es proteger los cofres y la puerta de salida, neutralizando simultáneamente al jugador. Para lograrlo, hemos garantizado que ningún agente centralice el comportamiento del sistema. El rol de "Gestor" en el protocolo Contract Net es dinámico: cualquier agente autónomo que vea al jugador iniciará la red de contratos, asumiendo la responsabilidad organizativa solo durante el tiempo que dura la subasta, devolviendo el control descentralizado al terminar.

La toma de decisiones global (reparto de tareas) busca optimizar el rendimiento del grupo adjudicando las tareas de defensa estática a los fantasmas más cercanos a los objetivos.

Finalmente, para evitar comportamientos repetitivos, hemos implementado una creencia global compartida (tiempoUltimaSubastaGlobal). Esta variable

actúa como un periodo de enfriamiento que evita subastas redundantes si el equipo acaba de formarse, garantizando un comportamiento grupal óptimo para la detección y captura del ladrón